

## Final Project Write-Up: Developing an "Access Denied Explainer" Tool for AWS

### Problem Statement

In AWS environments, "Access Denied" errors are commonly caused by misconfigured IAM permissions, restrictive Service Control Policies (SCPs), or bucket-level policies. Identifying the root cause of these errors can be a challenging task, often requiring considerable manual effort to diagnose and resolve without clear feedback.

### Proposed Solution

Our team proposed developing an "Access Denied Explainer" tool to tackle these challenges. The tool aims to automatically capture "Access Denied" events, analyze the contributing policies (IAM, SCP, or bucket policies), and offer insights that guide users in resolving the issues. Features include real-time email alerts for critical events and a built-in EventBridge and CloudWatch dashboard for monitoring historical data and trends.

### Architecture Diagram

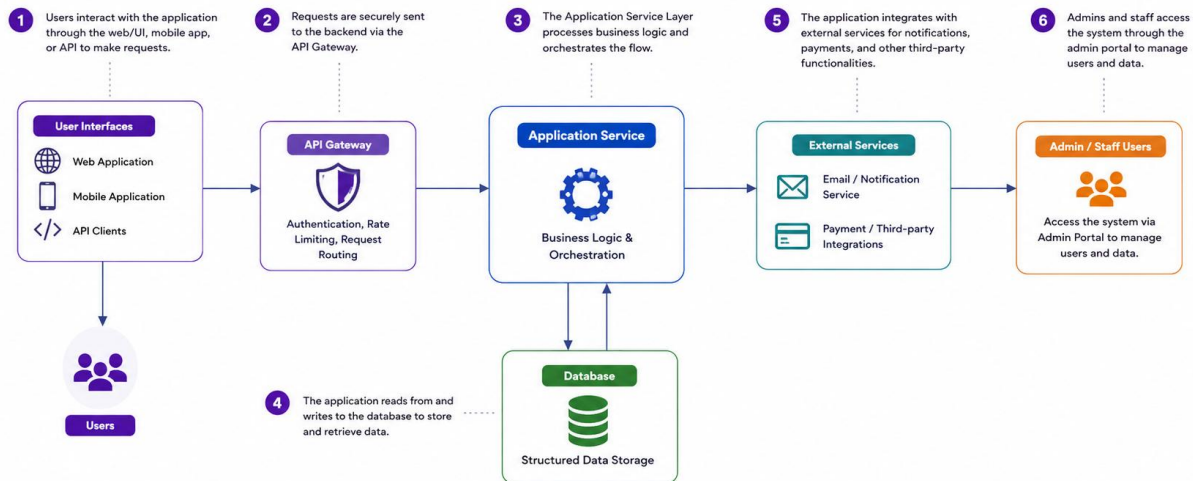


Figure 1-Architecture Diagram for the Solution

### Design Overview

The tool leverages several AWS services:

- **AWS CloudTrail:** Logs API activities, including access denial events.
- **AWS EventBridge:** Triggers Lambda functions in response to "Access Denied" events.
- **AWS Lambda:** Processes log data and identifies causes behind the access denial.

- **Amazon S3:** Stores logs for extended analysis.
- **Amazon CloudWatch:** Monitors logs for "Access Denied" patterns and triggers alarms based on preconfigured thresholds.
- **Amazon DynamoDB:** Provides a scalable backend for storing metadata and tracking historical data about access denial events.
- **AWS Secrets Manager:** Enables secure storage and management of sensitive data while enforcing strict access policies.

The solution emphasizes scalability, cost-efficiency, and user accessibility, with a focus on refining the tool iteratively to address evolving security requirements.

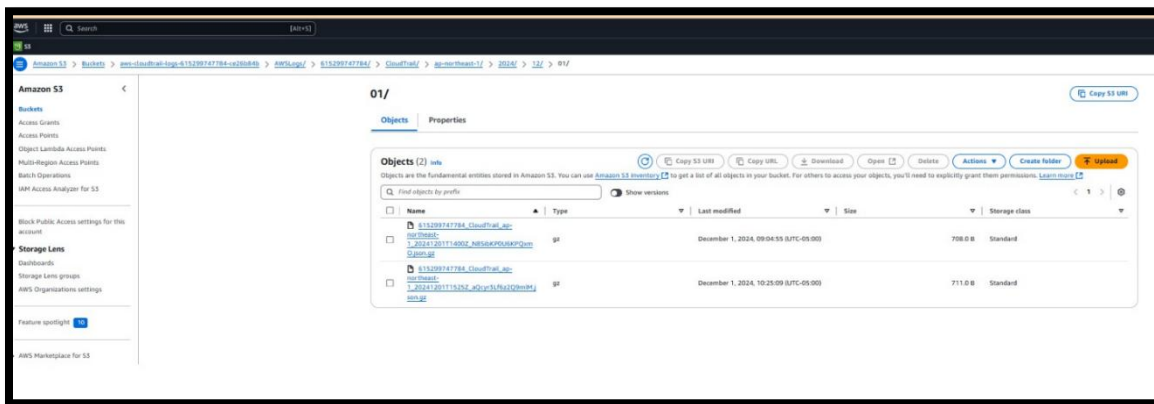


Figure 2-CloudTrail Recording API Activity including 'Access Denied' Errors

### Scenario 1: S3 Access Denied Alerts with KMS

This scenario focuses on using AWS Key Management Service (KMS) to secure S3 buckets and detect unauthorized access attempts. It integrates CloudTrail, CloudWatch, and SNS for real-time monitoring and notifications. Key Details are listed below:

- A symmetric key with restrictive policies ensures only authorized users can perform operations. Bucket policies mandate encryption using the specified KMS key.
- CloudTrail records API activity, while CloudWatch alarms are triggered for "Access Denied" events.
- SNS delivers detailed alerts, including the denied action, user identity, and impacted resources.

This setup highlights the role of KMS in enforcing strict access controls and facilitating proactive responses to unauthorized access attempts.

### Scenario 2: Real-Time Notifications of Access Denied Events and Secrets Managers

This scenario focuses on detecting and responding to "Access Denied" events specific to DynamoDB and AWS Secrets Manager resources. The solution integrates CloudTrail, EventBridge, Lambda, and SNS to enable real-time monitoring, processing, and notification.

- Detects unauthorized users attempting restricted operations like CreateTable in DynamoDB or CreateSecret in Secrets Manager.
- Routes "Access Denied" events from CloudTrail to Lambda functions for processing.
- Extracts event details such as user, operation, and error message, and sends alerts to an SNS topic.

This scenario highlights best practices for enhancing visibility into resource access, improving response time, and streamlining troubleshooting of access control issues in AWS environments.

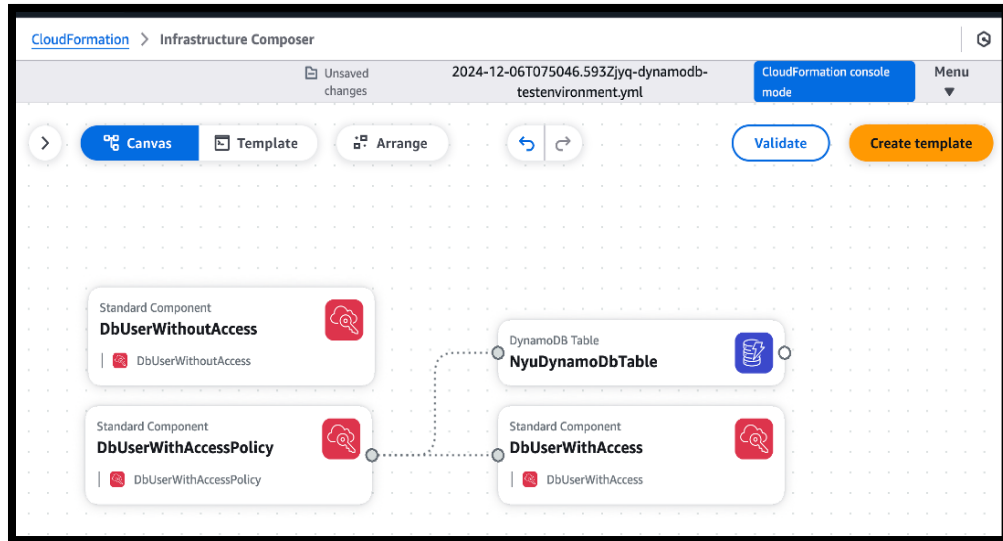


Figure 3-CloudFormation Template Visual For DynamoDB Scenario

## Issues Encountered

During implementation, we faced challenges with configuring AWS services. Setting up CloudTrail to capture detailed logs involved careful attention to IAM permissions. Creating EventBridge rules to detect specific denial patterns required precise tuning, and processing larger logs in Lambda led to performance issues. Configuring SNS email notifications also needed fine-tuning to ensure reliability. We attempted the integration of Lambda with the API Gateway which presented additional challenges.

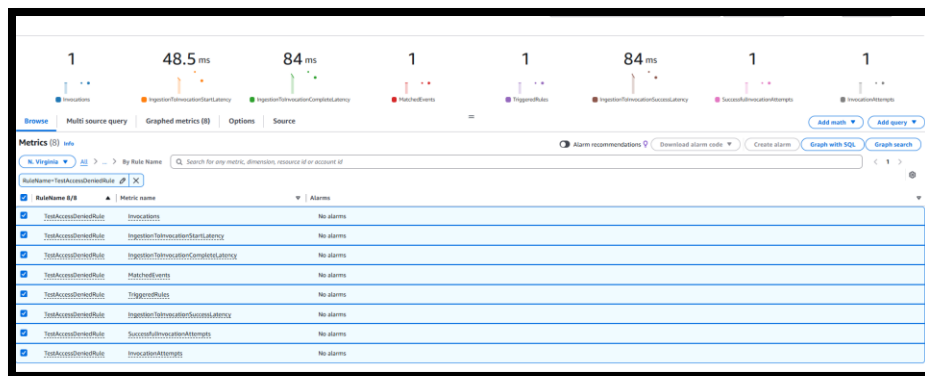


Figure 4-Amazon CloudWatch displaying various metrics including invocations, matched events and triggered rules

## **Final Design**

The final solution includes multiple AWS services working in tandem to address access denial issues effectively. AWS CloudTrail logs all API activities, especially "Access Denied" events, which are stored in Amazon S3 for future reference. EventBridge triggers the Lambda function for real-time event processing, and SNS notifications are sent out to ensure prompt action. The Lambda function provides users with specific insights about the cause of the access denial and potential fixes. Additionally, a monitoring dashboard on Amazon EventBridge and CloudWatch enables users to explore historical data, detect patterns, and track access denial events over time. Incorporating Secrets Manager adds another layer of security by ensuring sensitive data access is restricted to authorized users and enforcing policies to detect unauthorized access attempts. The design balances efficiency, scalability, and usability to meet our objectives.

## **Lessons Learned**

We learned that integrating AWS services such as CloudTrail, EventBridge, Lambda, and SNS requires patience and robust troubleshooting skills. Documentation played a key role in debugging and ensuring consistency. The iterative refinement approach was instrumental in developing a reliable tool, as continuous testing and adjustments resulted in a more user-friendly solution. Collaboration among team members proved to be essential for effective communication and workload distribution, which enabled us to meet our project goals.

## **Future Work**

There are several opportunities for enhancing the tool. Adding anomaly detection algorithms could enable automated identification of unusual patterns and potentially malicious activities, thereby improving security monitoring. Furthermore, extending the tool's capabilities to include other AWS error types and additional services like AWS Organizations or API Gateway would make it more versatile. Finally, integrating this tool with other security monitoring solutions would create a more comprehensive approach to access management and cloud security.